Chapter-6 (Ian Goodfellow)
KL Divergence, Cross-Entropy, Logistic Regression and SGD

Master's-level Deep Learning

Refai Arafat Rossi

October 2026

Your lesson can begin with two practical problems. The first explains how a neural network measures the mismatch between predicted and target probability distributions. The second explains how a model learns to improve its predictions by updating its parameters.

These examples connect Goodfellow's Sections 3.13, 5.7.1, 5.9, and 6.2.1 without moving into the other topics you plan to teach later. The equations and explanations below follow the book's information-theoretic and maximum-likelihood framework.

Problem 1 — How can a deep learning model learn the correct probability distribution?

1. The problem

Suppose we are developing a deep learning model to classify medical images into two categories: **Class A** and **Class B**.

For a simplified teaching example, assume that the target distribution represents uncertainty in the label:

Class	**Target probability** P	**Model prediction** Q
Class A	0.70	0.60
Class B	0.30	0.40
Total	**1.00**	**1.00**

The model predicts 60% for Class A and 40% for Class B, while the target distribution is 70% and 30%.

Our objective is to measure how different the model's prediction is from the target distribution and give the learning algorithm a loss to minimize.

Question for students: How can we measure this mismatch mathematically?

Goodfellow's Section 3.13 introduces two closely related measures:

- **Cross-entropy:** measures the average information cost incurred when using the model distribution Q to represent outcomes drawn from the target distribution P.
- **Kullback–Leibler (KL) divergence:** measures the extra information cost caused by the mismatch between P and Q.

Both use logarithms of probabilities. For the numerical calculations below, we use the natural logarithm, ln, so the results are expressed in nats.

2. Understand the three related terms

A. Entropy — uncertainty in the target distribution

Entropy is defined as:

$$H(P) = -\sum_i P(i) \log P(i)$$

It measures uncertainty in the target distribution itself. For a binary distribution $P = (0.7, 0.3)$, entropy is:

$$\begin{aligned} H(P) &= -[0.7 \ln(0.7) + 0.3 \ln(0.3)] \\ &= 0.6109 \text{ nats} \end{aligned}$$

B. Cross-entropy — mismatch measured using the model's probabilities

$$\boxed{H(P, Q) = -\sum_i P(i) \log Q(i)}$$

Here, the target probability $P(i)$ weights the logarithm of the probability assigned by the model, $Q(i)$.

C. KL divergence — extra information cost due to the mismatch

$$\boxed{D_{\mathrm{KL}}(P \| Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}}$$

KL divergence is non-negative and equals zero when the two distributions match. It is not a symmetric distance: exchanging P and Q generally changes its value.

3. The essential relationship

This is the most important equation for the first example:

$$\boxed{H(P, Q) = H(P) + D_{\mathrm{KL}}(P \| Q)}$$

In words: Cross-entropy = target entropy + KL divergence.

The target entropy $H(P)$ does not depend on the model parameters. Therefore, when training a model to match a fixed target distribution, minimizing cross-entropy is equivalent to minimizing KL divergence. This is the central relationship in Goodfellow's Section 3.13.

4. Solve the problem step by step

We have:

$$P = (0.7, 0.3), \qquad Q = (0.6, 0.4)$$

Step 1: Calculate cross-entropy.

$$\begin{aligned} H(P, Q) &= -[0.7 \ln(0.6) + 0.3 \ln(0.4)] \\ &= 0.3576 + 0.2749 \\ &= \boxed{0.6325 \text{ nats}} \end{aligned}$$

Step 2: Calculate KL divergence.

$$\begin{aligned} D_{\mathrm{KL}}(P\|Q) &= 0.7 \ln \frac{0.7}{0.6} + 0.3 \ln \frac{0.3}{0.4} \\ &= 0.1079 - 0.0863 \\ &= \boxed{0.0216 \text{ nats}} \end{aligned}$$

Step 3: Verify the relationship.

$$\begin{aligned} H(P) + D_{\mathrm{KL}}(P\|Q) &= 0.6109 + 0.0216 \\ &= \boxed{0.6325} \end{aligned}$$

The result agrees with the calculated cross-entropy.

Step 4: What if the model improves?
Suppose the model changes its prediction to match the target: $Q = (0.7, 0.3)$. Now, $D_{\mathrm{KL}}(P\|Q) = 0$ and $H(P, Q) = H(P) = 0.6109$ nats.

Model distribution Q	**Cross-entropy**	**KL divergence**
$(0.6, 0.4)$	0.6325	0.0216
$(0.7, 0.3)$	0.6109	0.0000

Table 1: Comparison of model predictions.

The target distribution is fixed at $P = (0.7, 0.3)$. When $Q = P$, the mismatch disappears, but the target entropy remains.

5. How does this apply to ordinary deep learning classification?

In conventional supervised classification, an example usually has a single known class rather than a soft target distribution. Suppose the true class is Class A. Its target distribution is one-hot:

$$P = (1, 0)$$

- If the model assigns Class A a probability of 0.70, then $H(P, Q) = -\ln(0.70) = 0.3567$.
- If the model improves its prediction to 0.95, then $H(P, Q) = -\ln(0.95) = 0.0513$.

The loss decreases because the model assigns a higher probability to the correct class.

For a one-hot target, the target entropy is zero. Consequently, cross-entropy equals KL divergence in this particular case:

$$H(P) = 0 \quad \Longrightarrow \quad H(P, Q) = D_{\mathrm{KL}}(P\|Q)$$

This distinction is useful when explaining why KL divergence and cross-entropy are related but are not always numerically identical.

6. Visualize why cross-entropy penalizes wrong predictions

In binary classification, let p be the probability assigned to Class 1. The loss depends on the actual label y.

- For a correct label $y = 1$: $\quad L(p) = -\ln p$
- For a correct label $y = 0$: $\quad L(p) = -\ln(1 - p)$

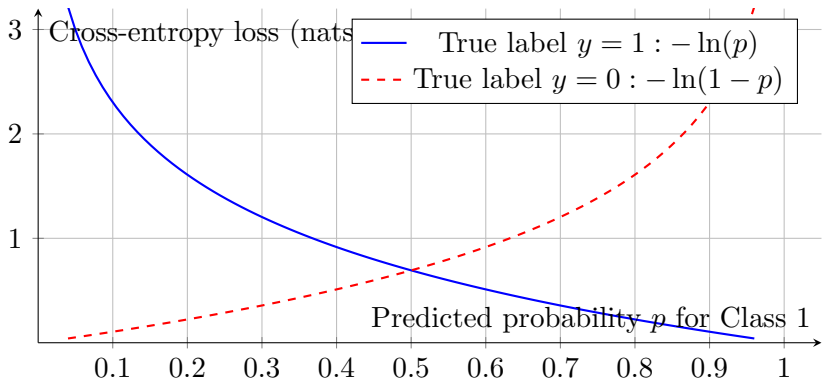


Figure 1: Cross-entropy loss curves as a function of predicted probability p.

For a true label of 1, assigning a probability near 1 produces a small loss. Assigning a probability near 0 produces a large loss. The reverse applies when the true label is 0.

This makes cross-entropy useful for training probabilistic classifiers: it rewards confident correct predictions and penalizes confident incorrect predictions.

Problem 2 — How can logistic regression learn using SGD?

Now we move from measuring prediction error to changing model parameters to reduce that error. This connects Goodfellow's Section 5.7.1 (Logistic Regression), Section 5.9 (Stochastic Gradient Descent), and Section 6.2.1 (Cost Functions).

1. The problem

Suppose we want to classify a network traffic sample as either:

- $y = 0$: benign traffic
- $y = 1$: malicious traffic

For this teaching example, we use just one numerical feature, x, representing a simplified suspicious-traffic score. The model receives $x = 2$, and the known label is $y = 1$, meaning malicious traffic.

Initially, the model has parameters:

$$w = 0, \qquad b = 0$$

Our objective is to use logistic regression and one SGD update to improve the prediction.

2. What is logistic regression?

Although its name contains regression, logistic regression is commonly used for binary classification. It estimates the probability that an input belongs to Class 1.

First, it calculates a linear score:

$$\boxed{z = wx + b}$$

- x is the input feature.

- w is the weight, controlling the effect of the feature.
- b is the bias, shifting the score.
- z is the linear score before the output activation.

The model then applies the sigmoid function:

$$\boxed{\sigma(z) = \frac{1}{1 + e^{-z}}}$$

Therefore,

$$\boxed{p = P(y = 1 \mid x) = \sigma(wx + b)}$$

The sigmoid converts any real-valued score into a probability strictly between 0 and 1. A common classification rule is:

$$\hat{y} = \begin{cases} 1, & p \geq 0.5 \\ 0, & p < 0.5 \end{cases}$$

The threshold determines the predicted class, but training normally uses the probability p, not just the final binary prediction.

3. Initial prediction

Given $x = 2$, $y = 1$, $w = 0$, $b = 0$:
First calculate the linear score:

$$z = wx + b = (0)(2) + 0 = 0$$

Apply sigmoid:

$$p = \frac{1}{1 + e^{0}} = \frac{1}{2} = 0.5$$

The model assigns a 50% probability to malicious traffic. This is an uncertain prediction. We now need a loss function to measure the error.

4. Use cross-entropy as the cost function

For binary classification, Goodfellow's maximum-likelihood framework gives the binary cross-entropy loss:

$$\boxed{L(y, p) = -[y \ln p + (1 - y) \ln(1 - p)]}$$

This is the negative log-likelihood of a Bernoulli output distribution, expressed as a loss to minimize. It is the binary form of the cross-entropy objective described in Section 6.2.1.

Because the true label is $y = 1$:

$$\begin{aligned} L &= -[1 \ln(0.5) + 0 \ln(0.5)] \\ &= -\ln(0.5) \\ &= \boxed{0.6931 \text{ nats}} \end{aligned}$$

The initial loss is 0.6931 nats. But how can the model reduce this loss? It must change its weight and bias.

5. Calculate the gradient

A gradient tells us how the loss changes when we change the model's parameters. It tells us which direction to move the parameters to reduce the loss locally.

For sigmoid output and binary cross-entropy, the derivative with respect to the linear score simplifies to:

$$\boxed{\frac{\partial L}{\partial z} = p - y}$$

Since $z = wx + b$, the chain rule gives:

$$\boxed{\frac{\partial L}{\partial w} = (p - y)x} \qquad \text{and} \qquad \boxed{\frac{\partial L}{\partial b} = p - y}$$

These gradients show the direction in which the weight and bias influence the loss.

Calculate the gradients using our example ($p = 0.5$, $y = 1$, and $x = 2$):

- **For the weight:**
$$\frac{\partial L}{\partial w} = (p - y)x = (0.5 - 1)(2) = \boxed{-1}$$
- **For the bias:**
$$\frac{\partial L}{\partial b} = p - y = 0.5 - 1 = \boxed{-0.5}$$

Both gradients are negative. To reduce the loss using gradient descent, we will increase both parameters.

6. Apply stochastic gradient descent (SGD)

Goodfellow's Section 5.9 describes SGD as updating the model parameters in the direction opposite to an estimated gradient:

$$\boxed{\theta \leftarrow \theta - \eta \hat{g}}$$

Here, θ represents all trainable parameters, $\hat{g}$ is the gradient estimate, and η is the learning rate.

Choose the learning rate $\eta = 0.1$:

- **Update the weight:**
$$w_{\text{new}} = w - \eta \frac{\partial L}{\partial w} = 0 - (0.1)(-1) = \boxed{0.1}$$
- **Update the bias:**
$$b_{\text{new}} = b - \eta \frac{\partial L}{\partial b} = 0 - (0.1)(-0.5) = \boxed{0.05}$$

The model parameters have changed from $w = 0, b = 0$ to $w = 0.1, b = 0.05$. Because the gradients are negative, subtraction increases the parameters.

7. Calculate the new prediction and loss

Using the updated parameters:

$$z_{\text{new}} = w_{\text{new}}x + b_{\text{new}} = (0.1)(2) + 0.05 = 0.25$$

Apply sigmoid:

$$p_{\text{new}} = \frac{1}{1 + e^{-0.25}} \approx \boxed{0.5622}$$

The predicted probability of malicious traffic has increased from 0.50 to approximately 0.5622.

Recalculate the loss:

$$L_{\text{new}} = -\ln(0.5622) \approx \boxed{0.5759 \text{ nats}}$$

	Loss (nats)	**Predicted attack probability**
Before the update	0.6931	50.00%
After the update	0.5759	56.22%

Table 2: Result of one SGD update.

The loss has decreased by approximately 0.1172 nats after this update. The model has moved toward predicting the correct class with a higher probability.

8. What is the relationship between logistic regression, cross-entropy and SGD?

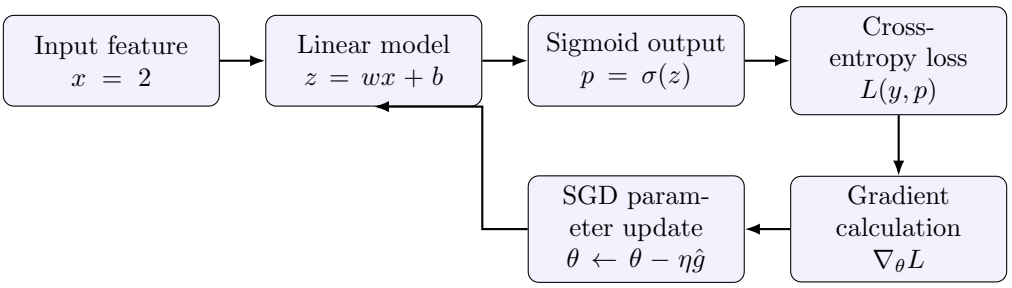


Figure 2: Computational pipeline for training logistic regression.

The roles are distinct:

- **Logistic regression** defines the prediction model: a linear score followed by sigmoid.
- **Cross-entropy** measures how well the predicted probability agrees with the true label.
- **Gradient calculation** determines how the trainable parameters affect the loss.
- **SGD** uses the gradient estimate to update the parameters.

Key message to give students: Cross-entropy tells the learning algorithm how wrong its probability predictions are. The gradient tells it how its parameters influence that error.

SGD then changes those parameters in an attempt to reduce the error.

Primary Textbook Reference: Goodfellow, Bengio and Courville, *Deep Learning*, MIT Press, 2016. The most relevant follow-up reading for these two examples is Section 6.2.1 (*Cost Functions*), which connects cross-entropy to maximum likelihood and model training.